Rapport d'activités

24/11/2025

Tom Planche

2025-2026

┌┬┬┐
│└ Remerciements 3
└ Introduction 4
  └ Affluences 4
└ Environnement de Travail 5
  ├ Conditions de travail et intégration 5
  └ Conclusion partielle 8
└ Missions 9
  └ Optimisation de requête 9

Chapter 1

Remerciements

Avant de commencer la lecture de ce mémoire, je tiens à adresser mes sincères remerciements aux personnes qui ont contribué au bon déroulement de mon année d'alternance et à la réalisation de ce document.

Je souhaite tout d'abord remercier l'entreprise Affluences pour m'avoir accueilli. J'ai particulièrement apprécié l'environnement de travail agréable et la liberté qui m'a été accordée dans le choix de mes outils de développement, me permettant de travailler dans des conditions optimales. Je remercie également l'ensemble de mes collègues pour leur accueil et la bonne ambiance générale.

Mes remerciements s'adressent tout particulièrement à mon maître d'apprentissage, Luis Valdes. Sa disponibilité constante, ses conseils avisés et son accompagnement m'ont été précieux tout au long de l'année. Son management bienveillant, alliant confiance et soutien, m'a permis de m'épanouir dans mes missions.

Je remercie également Micaël Pais Novo, CTO d'Affluences, pour sa disponibilité et la confiance qu'il m'a témoignée, me permettant de travailler en autonomie tout en sachant que je pouvais compter sur son aide.

Enfin, je souhaite exprimer ma gratitude à mes collègues pour leur aide précieuse. Merci à Jean-Charles Moussé pour son soutien sur le projet `app-api`, à Raphaël Galmiche pour son aide sur les déploiements, et à Justine Ribas pour ses éclaircissements sur `data-service`.

Je tiens aussi à remercier Nora Gabillé pour les nombreuses relectures et corrections de mes documents.

Chapter 2

Introduction

Ce rapport dresse le bilan de mon expérience en entreprise au cours de l'année scolaire 2025-2026. Ce parcours s'inscrit dans le cadre de ma formation d'ingénieur en informatique et systèmes d'information, réalisée en alternance au sein de l'École d'Ingénieur du Conservatoire National des Arts et Métiers (EI-CNAM)

J'ai donc intégré en alternance l'équipe backend d'Affluences, une entreprise française innovante spécialisée dans la gestion de l'affluence et l'optimisation des flux de visiteurs. J'intègre la partie **Internal Services** en tant que **développeur backend**.

I - Affluences

Affluences est une entreprise française fondée en 2014, aujourd'hui leader européen de la mesure et de la prévision d'affluence en temps réel. Sa mission est de transformer la gestion des flux de visiteurs en une expérience fluide et optimisée, tant pour les établissements que pour leurs usagers.

Avec plus de 2 000 lieux équipés et une application mobile utilisée par plus d'un million de personnes, Affluences a su prouver la pertinence de sa solution. L'entreprise a réalisé une levée de fonds de 4 millions d'euros en 2020 et compte parmi ses clients des institutions de renom comme le Musée du Louvre, la Tour Eiffel, la SNCF et l'Université de Cambridge.

La force d'Affluences réside dans sa solution technologique complète et intégrée, qui combine des capteurs IoT propriétaires pour la collecte de données, des algorithmes prédictifs pour anticiper les pics d'activité, et des plateformes de communication multi-canaux pour informer les utilisateurs en temps réel.

C'est au sein de cette `scale-up` innovante d'une cinquantaine de personnes, qui promeut une culture d'autonomie et d'actionnariat salarié, que j'ai eu l'opportunité de réaliser mon alternance.

Chapter 3

Environnement de Travail

Mon alternance s'est déroulée au sein d'un environnement de travail stimulant, caractérisé par une forte culture d'entreprise et une organisation agile et moderne. Cette section détaille les conditions de travail, l'environnement technique, et les méthodologies qui régissent le quotidien chez Affluences.

I - Conditions de travail et intégration

L'environnement de travail chez Affluences se distingue par une culture d'entreprise fondée sur la confiance, l'autonomie et la prise d'initiative. Avec une équipe d'une cinquantaine de personnes, l'organisation conserve une hiérarchie aplatie qui favorise la communication directe et la collaboration.

Un aspect particulièrement marquant de la culture d'Affluences est son modèle d'actionnariat salarié universel : chaque employé est actionnaire, ce qui aligne les intérêts de tous sur le succès collectif de l'entreprise. La transparence est également une valeur clé, avec une communication ouverte sur les résultats et la stratégie de l'entreprise.

L'intégration des nouveaux arrivants, et notamment des alternants, est facilitée par un système de mentorat et des perspectives d'évolution interne concrètes, illustrées par des parcours comme celui du Lead Mobile, qui a débuté en tant que stagiaire.

I.I - Environnement technique et outils

L'écosystème technique d'Affluences est riche et moderne, conçu pour supporter une plateforme traitant des millions d'utilisateurs et des centaines de millions de points de données annuellement.

I.I.1 - Stack technique principale

L'architecture backend repose sur une approche microservices, utilisant principalement **Node.js** et **Python** pour le traitement des données. La communication asynchrone entre les services est assurée par **RabbitMQ**. Côté frontend, les applications web s'appuient sur **Angular**, tandis que l'application mobile a été développée avec **Flutter**, le framework cross-platform de Google.

I.I.2 - Langages et frameworks

L'équipe de développement maîtrise un large éventail de langages et frameworks pour répondre aux besoins spécifiques de chaque partie de la plateforme :

- **Backend :** Node.js, Python.
- **Frontend :** JavaScript/TypeScript avec Angular, GraphQL pour les APIs.
- **Mobile :** Dart avec Flutter, avec une expérience passée sur le natif (Swift/Kotlin).

I.I.3 - Infrastructure et déploiement

L'infrastructure est hébergée sur le cloud français OVHcloud pour garantir la conformité RGPD. L'architecture distribuée s'appuie sur la conteneurisation avec **Docker**, probablement orchestrée par **Kubernetes**. Les bases de données suivent une approche multi-modèle, combinant probablement des bases de données relationnelles (SQL), NoSQL (pour les séries temporelles des capteurs) et un cache en mémoire comme **Redis** pour les données temps réel.

I.I.4 - Outils de productivité

Conformément à la culture d'autonomie, les développeurs ont la liberté de choisir leurs outils de travail, que ce soit leur système d'exploitation (OS) ou leur environnement de développement intégré (IDE), leur permettant de travailler dans des conditions de confort optimales.

I.I.5 - Sécurité et accès aux ressources

La sécurité est une priorité absolue. La plateforme est entièrement conforme au **Règlement Général sur la Protection des Données (RGPD)**, avec des mesures strictes d'anonymisation des données, de limitation de la durée de conservation et de chiffrement (SSL/TLS). En complément de cette conformité, l'entreprise est actuellement en démarche pour obtenir la certification **ISO/IEC 27001**, la norme internationale de référence pour les systèmes de management de la sécurité de l'information, afin de formaliser et d'attester de la robustesse de ses processus.

I.I.6 - Observabilité et monitoring

Le suivi de la plateforme en production est assuré par plusieurs outils. **Sentry** est utilisé pour le tracking d'erreurs en temps réel. Une solution d'APM (Application Performance Monitoring) et un système de logging centralisé sont également en place pour superviser la performance des microservices et faciliter le débogage.

I.I.7 - Outils collaboratifs

La collaboration est facilitée par la structure plate de l'entreprise et l'utilisation d'outils de communication modernes. Les réunions régulières comme les « Moments Affluences » et la transparence générale sur les objectifs permettent à chacun de comprendre sa contribution à la vision globale.

I.I.8 - Gestion des déploiements

Les déploiements sont automatisés via un pipeline de CI/CD (Intégration Continue / Déploiement Continu). Cette approche permet de livrer de nouvelles fonctionnalités de manière rapide et fiable, en s'assurant que chaque changement passe par une série de tests automatisés avant d'être mis en production. La stratégie de déploiement par phases, utilisée lors de la migration vers Flutter, illustre la maturité de ces processus.

I.II - Organisation du travail en mode agile

I.II.1 - Méthodologie de développement

Affluences a adopté une approche de développement agile rythmée par des sprints d'une semaine. Bien qu'un framework spécifique comme Scrum ne soit pas formellement appliqué dans toute sa rigueur, l'organisation du travail s'articule autour de cycles de développement itératifs et de rituels hebdomadaires bien établis.

Parmi ces rituels, on retrouve :

- Le `suivi-services`, qui se tient chaque lundi à 10h30. Cette réunion permet à chaque membre de l'équipe de partager ses avancées et les points de blocage éventuels.
- La `weekly tech`, qui a lieu le vendredi à 10h30. Ce point synchronise l'ensemble des équipes techniques (backend, frontend, mobile, data) et assure que chacun est informé des progrès et des défis des autres pôles.

Cette organisation, combinée à des équipes cross-fonctionnelles, permet de livrer de la valeur en continu tout en maintenant une forte cohésion et une bonne circulation de l'information au sein du département technique.

I.II.2 - Pipeline CI/CD

Le pipeline de CI/CD est au cœur de la méthodologie de développement. Il automatise la compilation, les tests et le déploiement du code, garantissant ainsi une haute qualité et une grande vélocité.

I.II.3 - Versionnage sémantique

L'équipe de développement suit les conventions du versionnage sémantique (SemVer) pour gérer les versions de ses applications et services. Cela permet de communiquer clairement l'impact des changements (corrections de bugs, nouvelles fonctionnalités, changements cassants) aux autres équipes et aux utilisateurs de l'API.

I.II.4 - Architecture orientée services

L'architecture microservices permet de découpler les différentes parties de la plateforme. Chaque service est responsable d'une fonctionnalité métier spécifique et peut être développé, déployé et mis à l'échelle indépendamment des autres. **RabbitMQ** joue un rôle crucial en permettant à ces services de communiquer de manière asynchrone et fiable.

I.II.5 - Standards de qualité

La qualité est assurée par une combinaison de revues de code systématiques, de tests automatisés (unitaires, intégration) intégrés au pipeline de CI/CD, et d'un monitoring proactif en production. La robustesse de l'architecture est conçue pour supporter une charge élevée tout en garantissant une haute disponibilité.

I.II.6 - Sécurité Docker

L'utilisation de Docker suit les meilleures pratiques de sécurité, notamment l'utilisation d'images de base minimalistes et vérifiées, la gestion des secrets en dehors des images, et potentiellement l'analyse des images pour détecter des vulnérabilités connues.

I.II.7 - Onboarding et documentation

L'intégration des nouveaux membres est une priorité. Le mentorat par des membres plus expérimentés de l'équipe est une pratique courante. La culture du partage de connaissances est également encouragée, notamment via le blog technique de l'entreprise qui sert de documentation sur les choix d'architecture et les défis techniques rencontrés.

I.II.8 - Communication et collaboration

La communication est fluide et directe grâce à la hiérarchie aplatie. Les équipes cross-fonctionnelles travaillent en étroite collaboration au quotidien. Les outils de messagerie instantanée et de gestion de projet viennent supporter ces échanges.

II - Conclusion partielle

L'environnement de travail chez Affluences est celui d'une `scale-up` technologique mature, qui a su conserver l'agilité et l'esprit d'initiative d'une startup tout en mettant en place des processus robustes pour garantir la qualité, la sécurité et la scalabilité de sa plateforme. La culture d'entreprise, axée sur l'autonomie, la transparence et l'intéressement collectif, constitue un atout majeur pour attirer et retenir les talents.

Chapter 4

Missions

I - Optimisation de requête

I.I - Contexte et problématique

La requête GraphQL `getAttendanceStatsForAPeriod` présentait des problèmes de performance critiques pour les plages de dates supérieures à un mois. Les requêtes prenaient plus de 90 secondes pour des périodes de 30 jours et crashaient complètement pour des requêtes sur une année entière.

Symptômes observés :

- Requête 30 jours : **90+ secondes**
- Requête 1 an : **Timeout** (crash complet)
- Dégradation exponentielle avec l'augmentation de la période
- Impact négatif sur l'expérience utilisateur des dashboards

I.II - Analyse technique de la cause racine

Le problème résidait dans l'architecture des requêtes du `AttendanceStatsRepository`, qui filtraient les données par `site_id` en utilisant l'extraction JSON :

```
WHERE JSON_EXTRACT(h.contextual_data, "$.site_id") = :siteId
  AND h.record_datetime_utc BETWEEN :from AND :to
  AND JSON_EXTRACT(h.data_points, "$.occupancy") IS NOT NULL
```

Definition 4.1. (Limitations de l'approche initiale):

- Impossibilité d'utiliser l'index de clé primaire `(measuring_set_id, record_datetime_utc)`
- Nécessité d'un parcours complet de la table (**full table scan**)
- Parsing JSON pour chaque ligne de la table
- Performance dégradant de manière exponentielle avec la taille de la période

I.III - Solution architecturale

L'optimisation a consisté à inverser la stratégie de requêtage pour exploiter l'indexation existante de la base de données.

Principe de la solution :

Approche initiale (lente) :
`site_id → [Scan complet + parsing JSON] → Résultats`

Nouvelle approche (optimisée) :
`site_id → [Appel API] → measuring_set_ids → [Requête indexée] → Résultats`

Au lieu de requêter directement par `site_id` (stocké dans un champ JSON), la solution procède en deux étapes :

1. **Récupération des measuring set IDs** via le `SensorsInternalHttpRepository`
2. **Requête par `measuring_set_id`** (colonne indexée) au lieu de `site_id` (champ JSON)

Cette approche ajoute un appel API léger (10-20ms) mais transforme la requête base de données de O(n) en O(log n).

I.IV - Modifications techniques implémentées

I.IV.1 - Refactoring du contrôleur

Le `AttendanceStatsController` a été rendu injectable avec injection de dépendances :

```typescript
@injectable()
export class AttendanceStatsController {
    constructor(private sensorsRepository: SensorsInternalRepository) {}

    private async getMeasuringSetIdsForSite(siteId: number): Promise<string[]> {
        const measuringSets = await this.sensorsRepository.getMeasuringSets({
            siteIds: [siteId],
            typeIn: ['IN_OUT_OCCUPANCY', 'ENTITY_COUNT_OCCUPANCY', 'WAITING_TIME'],
        });
        return measuringSets.map(ms => ms.measuringSetId);
    }
}
```

I.IV.2 - Optimisation des requêtes repository

Les signatures de méthodes ont été modifiées pour accepter des `measuring_set_ids` :

```typescript
public static async getMinMaxOccupancy(
    measuringSetIds: string[],
    fromDatetimeUtc: string,
    toDatetimeUtc: string
): Promise<{ maxOccupancy: number; minOccupancy: number } | null>
```

Les requêtes SQL ont été optimisées pour exploiter l'index :

```sql
WHERE h.measuring_set_id IN (:...measuringSetIds)
  AND h.record_datetime_utc BETWEEN :from AND :to
  AND h.data_points -> "$.occupancy" IS NOT NULL
```

Example 4.1. (Améliorations techniques):
- Utilisation de l'index de clé primaire
- Remplacement de `JSON_EXTRACT()` par l'opérateur `->` (plus lisible)
- Ajout de vérifications de nullité pour les tableaux vides
- Binding TypeORM d'arrays avec la syntaxe `:...array` pour les clauses `IN`

I.IV.3 - Injection de dépendances

Le pattern d'injection de dépendances a été correctement implémenté dans le conteneur IoC :

```
// app.module.ts
serviceContainer.bind<AttendanceStatsController>(AttendanceStatsController)
    .toSelf()
    .inSingletonScope();

// AttendanceStatsResolver.ts
@injectable()
export class AttendanceStatsResolver {
    constructor(private attendanceStatsController: AttendanceStatsController) {}
}
```

Cette approche améliore la testabilité et suit les patterns architecturaux existants du projet.

I.V - Résultats et impact

Gains de performance mesurés

Métrique	Avant	Après	Amélioration
Requête 30 jours	90+ secondes	40 ms	×2 250
Requête 1 an	Crash (timeout)	220 ms	$\infty \rightarrow$ 220 ms
Opération DB	Full table scan	Index seek	—
Comportement	Dégradation exp.	Performance linéaire	—

I.VI - Explication des optimisations clés

I.VI.1 - Exploitation de l'index composite

L'index de clé primaire `(measuring_set_id, record_datetime_utc)` est désormais pleinement exploité :

Definition 4.2. (Stratégie d'indexation):
- `measuring_set_id IN (...)` restreint aux partitions pertinentes
- `record_datetime_utc BETWEEN` utilise la seconde partie de l'index composite
- MySQL peut ignorer complètement les données non pertinentes

I.VI.2 - Trade-off et analyse coût-bénéfice

- **Coût ajouté** : 1 appel API pour récupérer les measuring set IDs (10-20 ms)
- **Coût économisé** : Élimination du scan complet avec parsing JSON (90+ secondes)
- **Gain net** : Amélioration de performance de ×2 250

I.VII - Impact en production

Cette optimisation a permis de :

- Rendre les dashboards réactifs pour des statistiques annuelles en temps réel
- Réduire drastiquement la charge sur la base de données
- Éliminer les erreurs de timeout pour les requêtes multi-mois
- Améliorer significativement l'expérience utilisateur avec des réponses instantanées

I.VIII - Conformité aux patterns existants

L'optimisation suit le même pattern déjà utilisé avec succès dans :

- `app/modules/attendance/infrastructure/record-history-mysql.repository.ts`
- `app/modules/attendance/services/attendance.service.ts`

La solution réutilise l'infrastructure existante (`SensorsInternalHttpRepository`) et respecte le pattern repository utilisé dans l'ensemble du codebase.

I.IX - Enseignements techniques

Leçons clés

1. **Conscience des index** : Toujours concevoir les requêtes autour des index disponibles
2. **Prudence avec les colonnes JSON** : Le filtrage sur des champs JSON empêche l'utilisation d'index
3. **Requêtes en deux étapes** : Ajouter une étape de lookup légère peut être plus rapide qu'une requête unique non optimisée
4. **Mesurer systématiquement** : L'amélioration de ×2 250 a été validée par des mesures en production réelle
5. **Suivre les patterns existants** : La solution réutilise l'architecture établie du projet

Concepts techniques approfondis

- **Index composites** : Compréhension du fonctionnement de `(measuring_set_id, record_datetime_utc)`
- **TypeORM array binding** : Syntaxe `:...array` pour les clauses `IN`
- **Opérateurs JSON MySQL** : Utilisation de `->` au lieu de `JSON_EXTRACT()`
- **Injection de dépendances** : Patterns IoC pour améliorer testabilité et maintenabilité

Example 4.2. (Fichiers modifiés):

- `app/controllers/attendance-stats/AttendanceStatsController.ts` — Ajout DI et résolution measuring sets
- `app/repositories/AttendanceStatsRepository.ts` — Optimisation requêtes avec colonnes indexées
- `app/resolvers/AttendanceStatsResolver.ts` — Injection contrôleur au lieu d'appels statiques
- `app/app.module.ts` — Enregistrement du contrôleur dans le conteneur IoC

Date de réalisation : Octobre 2025

Statut : ✅ Déployé en production et validé avec du trafic réel